

RQ2.R

Ester Maiocchi (10800862), Marianna Mazza (10838595), Arianna Perotti (10811649)

RQ2: Is a higher price justified by better accuracy and functionality or just by the brand's popularity? ----

```
df <- read.csv("df_clean.csv")
df <- df[,-1]
```

```
seed = 2002
```

Transform categorical variables into factors

```
df$Device_Name <- as.factor(df$Device_Name)
df$Brand <- as.factor(df$Brand)
df$Category <- as.factor(df$Category)
df$Model <- as.factor(df$Model)
```

Transform water resistance into a factor with ordered levels

```
unique(df$Water_Resistance_Rating)
```

```
## [1] "3ATM" "IP68" "IPX8" "5ATM" "IPX4" "IPX7" "10ATM"
```

```
df$Water_Resistance_Rating <- factor(df$Water_Resistance_Rating,
                                     levels = c("IPX4", "3ATM", "IPX7", "5ATM",
                                     "IPX8", "IP68", "10ATM"),
                                     ordered = TRUE)
```

```
df$Water_Resistance_Score <- as.numeric(df$Water_Resistance_Rating)
```

Transform connectivity into a factor with ordered levels

```
unique(df$Connectivity_Features)
```

```
## [1] "Bluetooth, WiFi" "WiFi, Bluetooth, NFC"
## [3] "Bluetooth" "WiFi, Bluetooth, NFC, LTE"
```

```
df$Connectivity_Features <- factor(
  df$Connectivity_Features,
  levels = c("Bluetooth", "Bluetooth, WiFi", "WiFi, Bluetooth, NFC", "WiFi,
Bluetooth, NFC, LTE"),
  ordered = TRUE)
```

```
df$Connectivity_Features_Score <- as.numeric(df$Connectivity_Features)
```

Transform app ecosystem into a factor with ordered levels

```
unique(df$App_Ecosystem_Support)
```

```
## [1] "Cross-platform" "iOS" "Android/iOS"
```

```
df$App_Ecosystem_Support <- factor(
  df$App_Ecosystem_Support,
  levels = c("iOS", "Android/iOS", "Cross-platform"),
```

```

ordered = TRUE)
df$App_Ecosystem_Support_Score <- as.numeric(df$App_Ecosystem_Support)

set.seed(2002)

library(dplyr)

##
## Caricamento pacchetto: 'dplyr'

## I seguenti oggetti sono mascherati da 'package:stats':
##
##     filter, lag

## I seguenti oggetti sono mascherati da 'package:base':
##
##     intersect, setdiff, setequal, union

library(ggplot2)
library(mgcv)      # GAM

## Caricamento del pacchetto richiesto: nlme

##
## Caricamento pacchetto: 'nlme'

## Il seguente oggetto è mascherato da 'package:dplyr':
##
##     collapse

## This is mgcv 1.9-3. For overview type 'help("mgcv-package")'.

library(quantreg) # Quantile regression

## Caricamento del pacchetto richiesto: SparseM

# Create a new variable where Lower GPS error becomes higher quality
df <- df %>%
  mutate(
    GPS_Quality = -GPS_Accuracy_Meters
  )

# List of performance-related variables to be used for ranking
vars_perf <- c(
  "Battery_Life_Hours",
  "Heart_Rate_Accuracy_Percent",
  "Step_Count_Accuracy_Percent",
  "Sleep_Tracking_Accuracy_Percent",
  "GPS_Quality",
  "Health_Sensors_Count",
  "Water_Resistance_Score",
  "Connectivity_Features_Score",

```

```

    "App_Ecosystem_Support_Score"
  )

# Check Linear correlation between overall Performance_Score and selected metrics
cor(df$Performance_Score, df$GPS_Accuracy_Meters)

## [1] 0.8946466

cor(df$Performance_Score, df$Heart_Rate_Accuracy_Percent)

## [1] -0.6953942

# Rank-transform selected performance variables (scaled between 0 and 1)
df_rank <- df %>%
  mutate(
    across(
      all_of(vars_perf),
      ~ rank(., na.last = "keep") / sum(!is.na(.)), # normalized rank
      .names = "r_{.col}" # prefix ranked variables with "r_"
    )
  )

# Construct composite indices based on ranked variables
df_rank <- df_rank %>%
  mutate(
    # Accuracy-related index (mean of ranked accuracy metrics)
    Accuracy_Index = rowMeans(
      select(., r_Heart_Rate_Accuracy_Percent,
              r_Step_Count_Accuracy_Percent,
              r_Sleep_Tracking_Accuracy_Percent,
              r_GPS_Quality),
      na.rm = TRUE
    ),
    # Functionality-related index (mean of ranked features metrics)
    Functionality_Index = rowMeans(
      select(., r_Battery_Life_Hours,
              r_Health_Sensors_Count,
              r_Water_Resistance_Score,
              r_Connectivity_Features_Score,
              r_App_Ecosystem_Support_Score),
      na.rm = TRUE
    ),
    # Overall performance index (equal weighting of accuracy and functionality)
    Performance_Index =
      0.5 * Accuracy_Index + 0.5 * Functionality_Index
  )

summary(df_rank$Performance_Index)

##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.1595  0.4083  0.5234  0.5002  0.5970  0.8424

```

```
# Correlation between new Performance_Index and the original Performance_Score (sign reversed)
```

```
cor(df_rank$Performance_Index,  
    -df$Performance_Score,  
    use = "complete.obs")
```

```
## [1] 0.5495554
```

```
# Excluding the "Fitness Band" category since it always has the same price
```

```
df_price <- df_rank %>%  
  filter(Category != "Fitness Band")
```

```
# Full GAM model: Price ~ smooth(Performance) + Brand ----
```

```
library(mgcv)
```

```
gam_price <- gam(  
  Price_USD ~ s(Performance_Index) + Brand,  
  data = df_price  
)
```

```
summary(gam_price)
```

```
##
```

```
## Family: gaussian
```

```
## Link function: identity
```

```
##
```

```
## Formula:
```

```
## Price_USD ~ s(Performance_Index) + Brand
```

```
##
```

```
## Parametric coefficients:
```

```
##           Estimate Std. Error t value Pr(>|t|)  
## (Intercept)    180.36      10.01  18.018 <2e-16 ***  
## BrandApple     347.09      14.03  24.742 <2e-16 ***  
## BrandFitbit     24.89      13.72   1.813  0.0699 .  
## BrandGarmin    393.84      15.72  25.054 <2e-16 ***  
## BrandHuawei    288.98      14.73  19.615 <2e-16 ***  
## BrandOura     245.70      14.43  17.032 <2e-16 ***  
## BrandPolar    163.69      14.00  11.694 <2e-16 ***  
## BrandSamsung   261.09      14.25  18.323 <2e-16 ***  
## BrandWithings  174.55      14.31  12.198 <2e-16 ***
```

```
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
```

```
## Approximate significance of smooth terms:
```

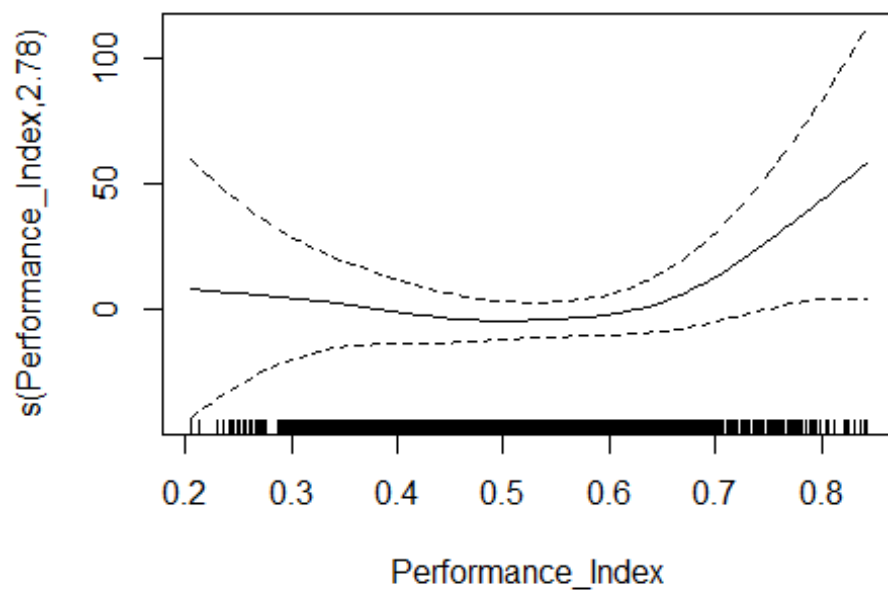
```
##           edf Ref.df      F p-value  
## s(Performance_Index) 2.778  3.555 1.507  0.227
```

```
##
```

```
## R-sq.(adj) = 0.435   Deviance explained = 43.8%
```

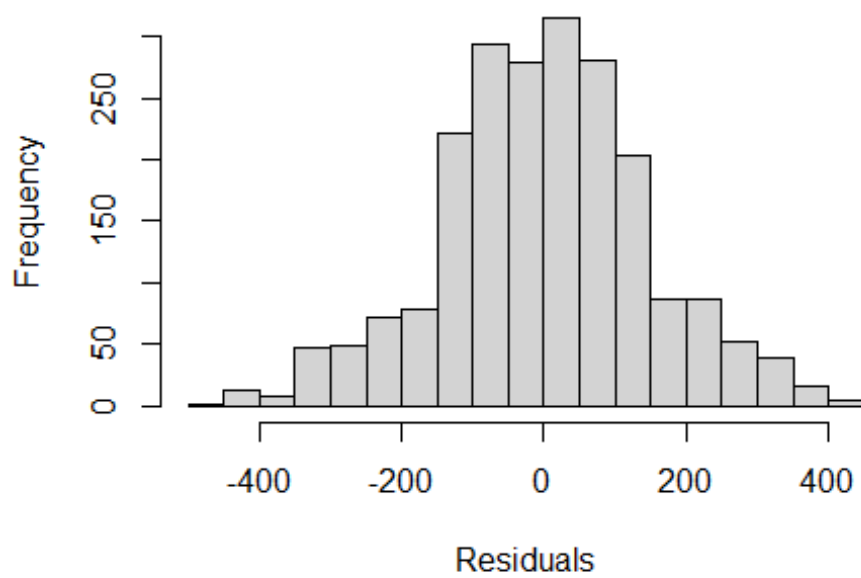
```
## GCV = 21909   Scale est. = 21789      n = 2144
```

```
plot(gam_price)
```



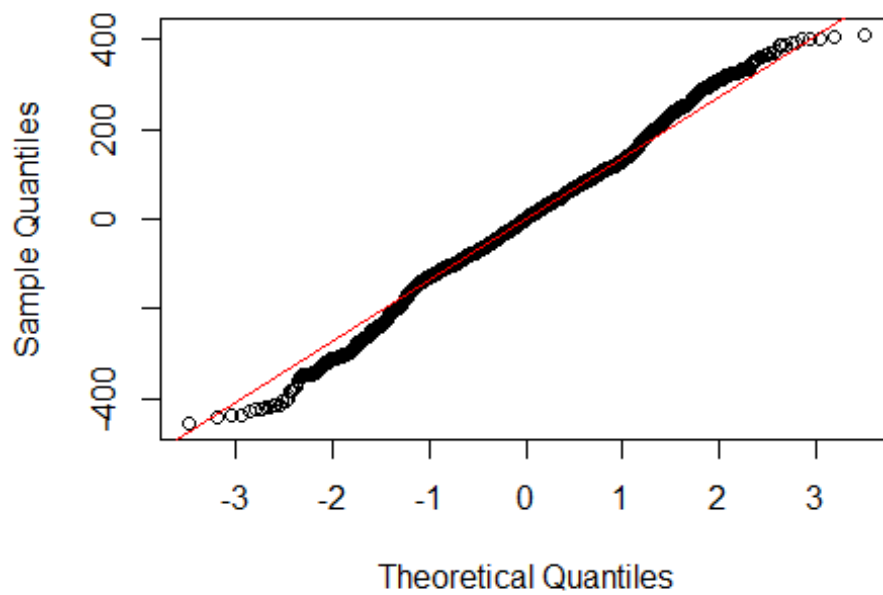
```
# Residual diagnostics  
res <- residuals(gam_price)  
hist(res, breaks = 30, main = "Histogram of GAM residuals",  
      xlab = "Residuals")
```

Histogram of GAM residuals



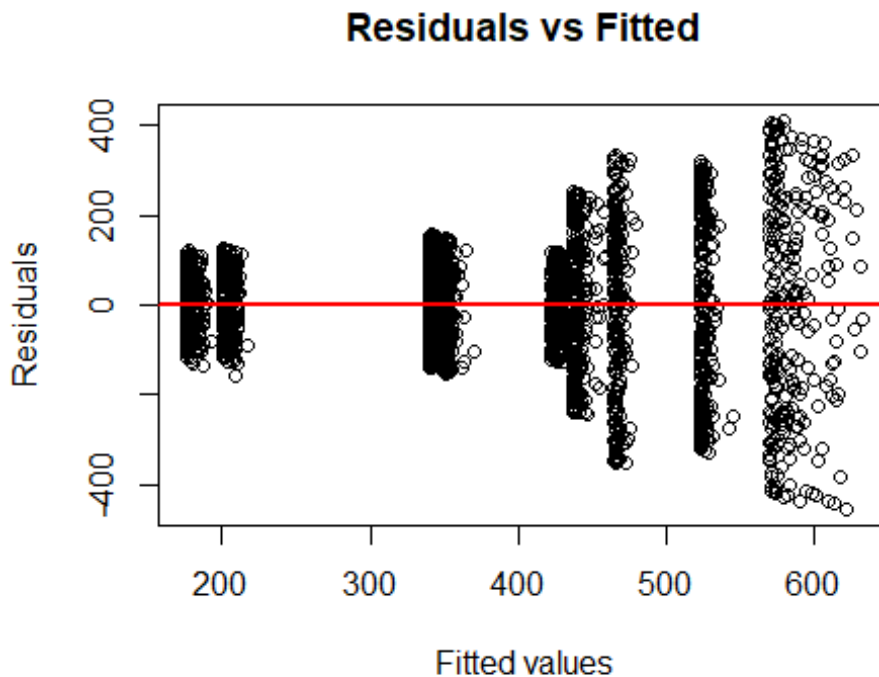
```
qqnorm(res)  
qqline(res, col = "red")
```

Normal Q-Q Plot



```
fit <- fitted(gam_price)
```

```
plot(
  fit, res,
  xlab = "Fitted values",
  ylab = "Residuals",
  main = "Residuals vs Fitted"
)
abline(h = 0, col = "red", lwd = 2)
```



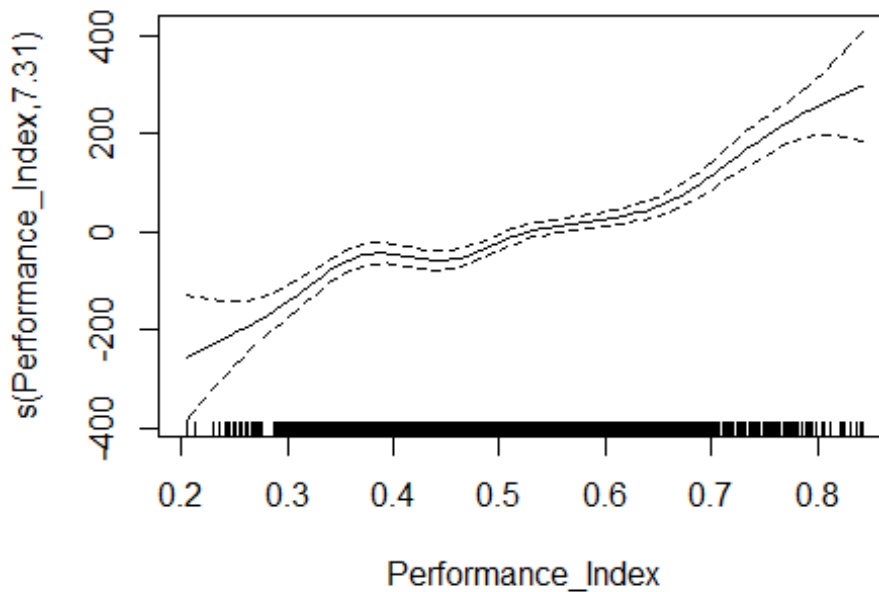
```
# GAM without brand: Price ~ smooth(Performance) ----
gam_perf_only <- gam(
  Price_USD ~ s(Performance_Index),
  data = df_price
)

summary(gam_perf_only)

##
## Family: gaussian
## Link function: identity
##
## Formula:
## Price_USD ~ s(Performance_Index)
##
## Parametric coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  394.940      3.992   98.92  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
## Approximate significance of smooth terms:
##               edf Ref.df      F p-value
## s(Performance_Index) 7.31  8.329 33.52  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.114   Deviance explained = 11.7%
## GCV =  34306   Scale est. = 34173       n = 2144

plot(gam_perf_only)
```

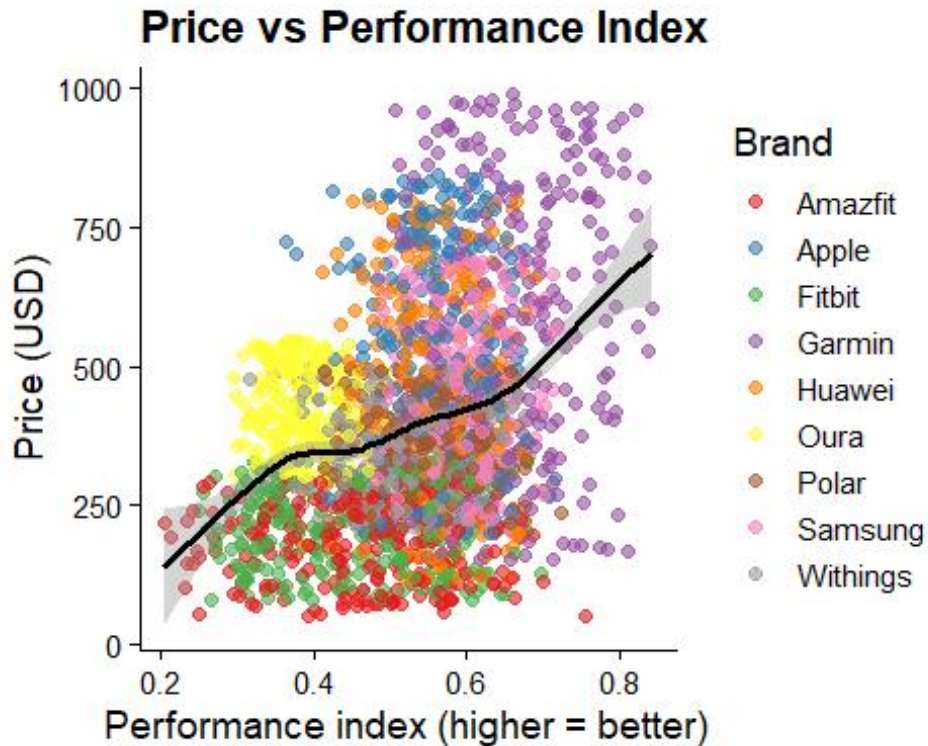


```
ggplot(df_price, aes(Performance_Index, Price_USD)) +
  geom_point(
    aes(color = Brand),
    alpha = 0.6,
    size = 2
  ) +
  geom_smooth(
    method = "gam",
    formula = y ~ s(x),
    color = "black",
    linewidth = 1.2,
    se = TRUE
  ) +
  scale_color_brewer(palette = "Set1") +
  labs(
    title = "Price vs Performance Index",
```

```

x = "Performance index (higher = better)",
y = "Price (USD)",
color = "Brand"
) +
theme_classic(base_size = 14) +
theme(
  legend.position = "right",
  plot.title = element_text(face = "bold")
)

```



```

# Quantile regression ----
library(quantreg)

# Fit quantile regression at multiple quantiles
rq_perf <- rq(
  Price_USD ~ Performance_Index,
  tau = c(0.05, 0.25, 0.5, 0.75, 0.95),
  data = df_price
)

summary(rq_perf)

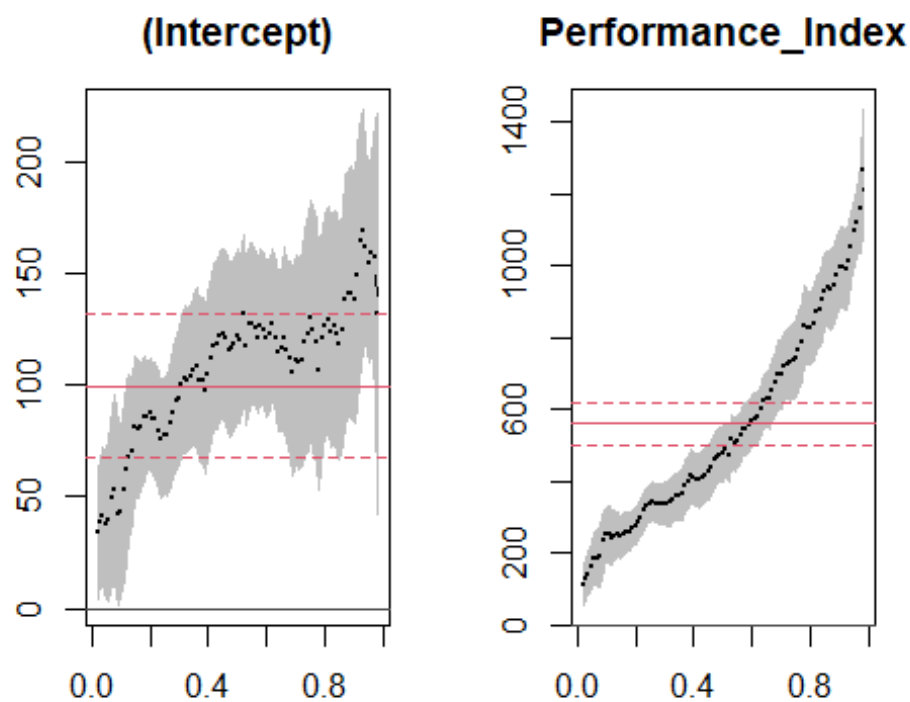
##
## Call: rq(formula = Price_USD ~ Performance_Index, tau = c(0.05, 0.25,
##      0.5, 0.75, 0.95), data = df_price)
##
## tau: [1] 0.05
##

```

```

## Coefficients:
##               Value      Std. Error t value   Pr(>|t|)
## (Intercept)    37.55972    20.27754   1.85228   0.06412
## Performance_Index 162.74971    39.16170   4.15584   0.00003
##
## Call: rq(formula = Price_USD ~ Performance_Index, tau = c(0.05, 0.25,
##      0.5, 0.75, 0.95), data = df_price)
##
## tau: [1] 0.25
##
## Coefficients:
##               Value      Std. Error t value   Pr(>|t|)
## (Intercept)    77.47435    16.13471   4.80172   0.00000
## Performance_Index 335.73107    32.08348  10.46430   0.00000
##
## Call: rq(formula = Price_USD ~ Performance_Index, tau = c(0.05, 0.25,
##      0.5, 0.75, 0.95), data = df_price)
##
## tau: [1] 0.5
##
## Coefficients:
##               Value      Std. Error t value   Pr(>|t|)
## (Intercept)    121.71481    23.50563   5.17811   0.00000
## Performance_Index 475.06210    47.27107  10.04974   0.00000
##
## Call: rq(formula = Price_USD ~ Performance_Index, tau = c(0.05, 0.25,
##      0.5, 0.75, 0.95), data = df_price)
##
## tau: [1] 0.75
##
## Coefficients:
##               Value      Std. Error t value   Pr(>|t|)
## (Intercept)    130.04871    31.95690   4.06950   0.00005
## Performance_Index 741.46104    64.51973  11.49200   0.00000
##
## Call: rq(formula = Price_USD ~ Performance_Index, tau = c(0.05, 0.25,
##      0.5, 0.75, 0.95), data = df_price)
##
## tau: [1] 0.95
##
## Coefficients:
##               Value      Std. Error t value   Pr(>|t|)
## (Intercept)    154.32069    26.78095   5.76233   0.00000
## Performance_Index 1095.60361    56.24622  19.47870   0.00000
##
# Plot coefficient estimates across 2nd to 98th percentiles
plot(summary(rq(Price_USD ~ Performance_Index, tau=2:98/100, data = df_price)),
mfrow = c(1,2))

```



```
rq_05 <- rq(Price_USD ~ Performance_Index, tau = 0.05, data = df_price)
rq_25 <- rq(Price_USD ~ Performance_Index, tau = 0.25, data = df_price)
rq_50 <- rq(Price_USD ~ Performance_Index, tau = 0.50, data = df_price)
rq_75 <- rq(Price_USD ~ Performance_Index, tau = 0.75, data = df_price)
rq_95 <- rq(Price_USD ~ Performance_Index, tau = 0.95, data = df_price)
```

```
# Scatter plot with fitted quantile regression lines
plot(df_price$Performance_Index, df_price$Price_USD,
     main = "Quantile Regression: Performance vs Price",
     xlab = "Performance Index (higher = better)",
     ylab = "Price (USD)",
     pch = 16,
     col = rgb(0, 0, 0, 0.4))
```

```
# Add regression Lines for each quantile
abline(a = coef(rq_05)[1],
       b = coef(rq_05)["Performance_Index"],
       col = "orange", lty = 2, lwd = 2)
```

```
abline(a = coef(rq_25)[1],
       b = coef(rq_25)["Performance_Index"],
       col = "red", lty = 2, lwd = 2)
```

```
abline(a = coef(rq_50)[1],
       b = coef(rq_50)["Performance_Index"],
       col = "blue", lty = 2, lwd = 2)
```

```

abline(a = coef(rq_75)[1],
       b = coef(rq_75)["Performance_Index"],
       col = "green", lty = 2, lwd = 2)

abline(a = coef(rq_95)[1],
       b = coef(rq_95)["Performance_Index"],
       col = "purple", lty = 2, lwd = 2)

legend("topleft",
       legend = c("Quantile 0.05", "Quantile 0.25", "Quantile 0.50", "Quantile
0.75", "Quantile 0.95"),
       col = c("orange", "red", "blue", "green", "purple"),
       lty = 2,
       lwd = 2,
       bty = "n")

```

Quantile Regression: Performance vs Price

